Batch vs. Stream Processing

Explore the core concepts of batch and stream processing for ETL pipelines. Understand how each method handles data, their typical applications, and the critical performance metrics of latency and throughput. This lesson helps you decide the correct processing type based on system needs and user expectations.

There are two types of processing techniques when building data pipelines for delivering data from a source to a destination:

- Batch processing
- Stream processing

Before discussing each, it's important to mention the types of systems we can find today, how users interact with them, and how they process their data. We'll also discuss the two major metrics for measuring pipeline performance: latency and throughput.

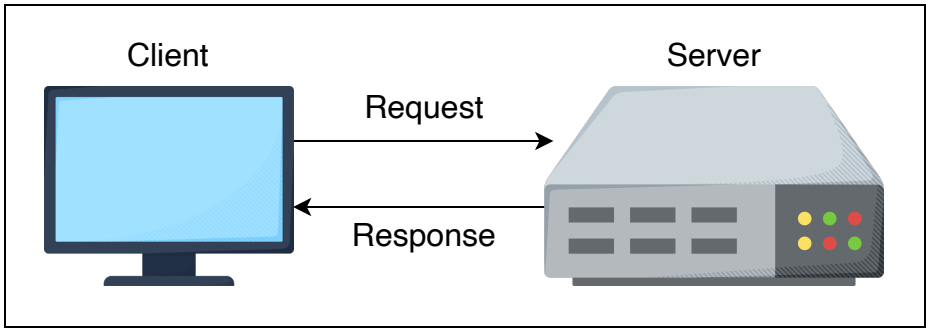
Online systems

Today, it's very common to find systems where we ask for something or send some instruction, and the output is returned a short time later. This is how most of the systems we interact with today work, including caches, search engines, databases, and web servers.

Online systems use a client-server architecture. They consist of one or more servers that host services. These services can send information over the in to clients with the required authorization. In most cases, the client is a human making a service request and waiting for the response to arrive from a server.

Ask

They're called online systems because they constantly wait for a request from a client.



Online systems—Client/server architecture

Batch processing systems

Compared to the small packets of data that an online system needs to process, a **batch processing system** will take much larger input data, process it for much longer, and retrieve an output.

In these systems, a human doesn't make the request and then waits for a response. The process is scheduled to run periodically every certain amount of time, usually after working hours, and the output will most likely be stored in a non-volatile repository.

Data pipelines that process batches of data (like the ETL we just built) can be considered batch processing systems. These pipelines are used mostly for analytics because there's no need for real-time data availability and no user waiting for output on the other side.

Ask

Batch data pipeline

This is the core difference between batch and streaming. If no human is on the other side waiting for the pipeline to finish, we can schedule the pipeline to run when the system's load is expected to be low. The main performance metric in batch processing pipelines is **throughput**.

Some use cases for batch processing are:

- Transferring data to a backup repository

Ask

- Storing historical data for BI and analytics
- Processing financial data for auditing or billing

Stream processing systems

Stream processing systems fall somewhere between the previous two. These systems operate on events shortly after they happen. They'll most likely collect a small number of events according to a predefined amount or a fixed amount of time passed, then, process them and send output. These systems will sometimes have a human waiting for an output to arrive.

Data pipelines that process streams of data can be considered **stream processing systems**. These pipelines operate on very small packets of data. The idea is to deal with packets that are so small that we can process them immediately and continuously deliver them to the destination.

Ask

Stream data pipeline

Stream data pipelines are used mostly by applications and systems that need constantly available data, such as analytics, monitoring, and alerting. They can feed machine learning and AI models that make decisions in real-time, autonomous vehicles, or recommendation engines such as Netflix. This is why the main performance metric for stream processing pipelines is latency.

Stream processing is more complex than batch processing. Remember, our needs determine the choice of method. If we don't need constant data processing in real time with low latency, we should use batch processing. It's simpler to build, monitor, and configure.

Latency vs. throughput

The two most important metrics when discussing data pipelines are latency and throughput.

Data pipeline performance metrics: Latency and throughput

Latency is the time it takes for a single packet of data to pass through the pipeline. Because a pipeline is composed of many processes working together in sequence, the slowest process in the pipeline limits the overall latency of t entire pipeline and is called the **bottleneck**. Latency is an extremely important metric when building pipelines that transfer data to near-real-time systems. The

Ask

simple reason is that people don't like to wait much for their output to arrive. For example, a pipeline that processes financial data using a machine learning model that detects fraudulent transactions and alerts the users will need near-real-time performance and very low latency.

Throughput is the amount of data a pipeline can process in a unit of time. The

Ask